dYdX v4 Indexer 데이터베이스 스키마 분석

개요

dYdX v4 Indexer는 PostgreSQL을 기반으로 한 관계형 데이터베이스를 사용하여 온체인 및 오프체인 데이터를 저장하고 조회합니다. 이 문서는 Indexer의 데이터베이스 스키마, 테이블 구조, 인덱싱 전략, 그리고 성능 최적화 방법을 상세히 분석합니다.

핵심 테이블 구조

1. Subaccounts 테이블

테이블 정의

```sql
CREATE TABLE subaccounts (
id UUID PRIMARY KEY,
address VARCHAR NOT NULL,
subaccountNumber INTEGER NOT NULL,
updatedAt TIMESTAMP NOT NULL,
updatedAtHeight BIGINT NOT NULL
);

-- 인덱스
CREATE INDEX ON subaccounts (address);
```

```
CREATE INDEX ON subaccounts (updatedAt);
```

필드 분석

- **id**: UUID 기본 키, 서브계정의 고유 식별자
- **address**: Cosmos 지갑 주소 (bech32 형식)
- **subaccountNumber**: 계정 번호 (0-127,999 범위)
- **updatedAt**: 마지막 업데이트 시간
- **updatedAtHeight**: 마지막 업데이트 블록 높이

관계 매핑

```
static relationMappings = {
orders: {
relation: Model.HasManyRelation,
modelClass: 'order-model',
join: { from: 'subaccounts.id', to: 'orders.subaccountId' }
},
perpetualPositions: {
relation: Model.HasManyRelation,
modelClass: 'perpetual-position-model',
join: { from: 'subaccounts.id', to: 'perpetual_positions.subaccountId' }
},
assetPositions: {
relation: Model.HasManyRelation,
```

```
modelClass: 'asset-position-model',

join: { from: 'subaccounts.id', to: 'asset_positions.subaccountId' }

}

};
```

2. Orders 테이블

테이블 정의

```
CREATE TABLE orders (

id UUID PRIMARY KEY,

subaccountId UUID NOT NULL REFERENCES subaccounts(id),

clientId BIGINT NOT NULL,

clobPairId BIGINT NOT NULL,

side ENUM('BUY', 'SELL') NOT NULL,

size DECIMAL NOT NULL,

totalFilled DECIMAL NOT NULL,

price DECIMAL NOT NULL,

type ENUM('LIMIT', 'MARKET', 'STOP_LIMIT', 'STOP_MARKET',

'TRAILING_STOP', 'TAKE_PROFIT', 'TAKE_PROFIT_MARKET',

'LIQUIDATED', 'LIQUIDATION', 'HARD_TRADE',

'FAILED_HARD_TRADE', 'TRANSFER_PLACEHOLDER') NOT NULL,

status ENUM('OPEN', 'FILLED', 'CANCELED',

'BEST_EFFORT_CANCELED', 'UNTRIGGERED') NOT NULL,
```

```
timeInForce ENUM('GTT', 'FOK', 'IOC', 'POST_ONLY') NOT NULL,
reduceOnly BOOLEAN  NOT NULL,
orderFlags BIGINT  NOT NULL,
goodTilBlock BIGINT  NULL,
goodTilBlockTime TIMESTAMP  NULL,
createdAtHeight BIGINT  NULL,
clientMetadata BIGINT  NOT NULL,
triggerPrice DECIMAL  NULL,
updatedAt TIMESTAMP  NOT NULL,
updatedAtHeight BIGINT  NOT NULL
);
```

인덱싱 전략

```
-- 기본 인덱스
CREATE  INDEX  ON orders (subaccountId);
CREATE  INDEX  ON orders (clobPairId, side, price);

-- 복합 인덱스 (성능 최적화)
CREATE  INDEX  ON orders (clobPairId, subaccountId);
CREATE  INDEX  ON orders (subaccountId, status);
CREATE  INDEX  ON orders (createdAtHeight, transactionIndex);
```

주문 타입 및 상태

```
export enum OrderType {
LIMIT = 'LIMIT', // 지정가 주문
MARKET = 'MARKET', // 시장가 주문
STOP_LIMIT = 'STOP_LIMIT', // 스탑 지정가
STOP_MARKET = 'STOP_MARKET', // 스탑 시장가
TRAILING_STOP = 'TRAILING_STOP', // 트레일링 스탑
TAKE_PROFIT = 'TAKE_PROFIT', // 이익 실현
LIQUIDATED = 'LIQUIDATED', // 청산된 주문
LIQUIDATION = 'LIQUIDATION'  // 청산 주문
}

export enum OrderStatus {
OPEN = 'OPEN', // 활성 주문
FILLED = 'FILLED', // 완전 체결
CANCELED = 'CANCELED', // 취소됨
BEST_EFFORT_CANCELED = 'BEST_EFFORT_CANCELED', // 최선 노력 취소
UNTRIGGERED = 'UNTRIGGERED'  // 미트리거 (조건부 주문)
}
```

3. Fills 테이블

테이블 정의

```
CREATE TABLE fills (
```

```
id UUID PRIMARY KEY,
subaccountId UUID NOT NULL  REFERENCES subaccounts(id),
side ENUM('BUY', 'SELL') NOT NULL,
liquidity ENUM('TAKER', 'MAKER') NOT NULL,
type ENUM('LIMIT', 'LIQUIDATED', 'LIQUIDATION',
'DELEVERAGED', 'OFFSETTING') NOT NULL,
clobPairId BIGINT  NOT NULL,
orderId UUID NULL  REFERENCES orders(id),
size  DECIMAL  NOT NULL,
price DECIMAL  NOT NULL,
quoteAmount DECIMAL  NOT NULL,
eventId BYTEA  NOT NULL,
transactionHash VARCHAR  NOT NULL,
createdAt TIMESTAMP  NOT NULL,
createdAtHeight BIGINT  NOT NULL,
clientMetadata BIGINT  NULL,
fee DECIMAL  NOT NULL
);
```

성능 최적화 인덱스

```
-- 기본 조회 인덱스
CREATE  INDEX  ON fills (subaccountId, createdAt);
```

```
CREATE INDEX ON fills (subaccountId, createdAtHeight);

CREATE INDEX ON fills (subaccountId, clobPairId);

CREATE INDEX ON fills (clobPairId, createdAt);

CREATE INDEX ON fills (orderId);


-- 부분 인덱스 (TAKER 체결만)

CREATE INDEX fills_clobPairId_createdAtHeight_partial

ON fills (clobPairId, createdAtHeight)

WHERE liquidity = 'TAKER';


-- 복합 부분 인덱스

CREATE INDEX fills_clobPairId_createdAtHeight_createdAt_partial

ON fills (clobPairId, createdAtHeight, createdAt)

WHERE liquidity = 'TAKER';
```

체결 타입 분석

```
export enum FillType {

LIMIT = 'LIMIT', // 일반 지정가 체결

LIQUIDATED = 'LIQUIDATED', // 청산당한 계정의 체결 (테이커)

LIQUIDATION = 'LIQUIDATION', // 청산 실행 체결 (메이커)

DELEVERAGED = 'DELEVERAGED', // 디레버리징된 계정 (테이커)

OFFSETTING = 'OFFSETTING'  // 디레버리징 상대방 (메이커)

}
```

```
export enum Liquidity {
TAKER = 'TAKER', // 유동성 소비자
MAKER = 'MAKER' // 유동성 공급자
}
```

4. Perpetual Positions 테이블

테이블 정의

```
CREATE TABLE perpetual_positions (
id UUID PRIMARY KEY,
subaccountId UUID NOT NULL REFERENCES subaccounts(id),
perpetualId BIGINT NOT NULL REFERENCES perpetual_markets(id),
side ENUM('LONG', 'SHORT') NOT NULL,
status ENUM('OPEN', 'CLOSED', 'LIQUIDATED') NOT NULL,
size DECIMAL NOT NULL,
maxSize DECIMAL NOT NULL,
entryPrice DECIMAL NOT NULL,
exitPrice DECIMAL NULL,
sumOpen DECIMAL NOT NULL,
sumClose DECIMAL NOT NULL,
createdAt TIMESTAMP NOT NULL,
closedAt TIMESTAMP NULL,
```

```
createdAtHeight BIGINT  NOT NULL,
closedAtHeight BIGINT  NULL,
openEventId BYTEA  NOT NULL,
closeEventId BYTEA  NULL,
lastEventId BYTEA  NOT NULL,
settledFunding DECIMAL  NOT NULL
);
```

포지션 관련 인덱스

```
-- 기본 조회 인덱스
CREATE  INDEX  ON perpetual_positions (subaccountId, status);
CREATE  INDEX  ON perpetual_positions (subaccountId, createdAt);
CREATE  INDEX  ON perpetual_positions (perpetualId, status);

-- 복합 인덱스 (성능 최적화)
CREATE  INDEX  perpetual_positions_subaccount_perpetual_created_at_index
ON perpetual_positions (subaccountId, createdAtHeight, perpetualId);

CREATE  INDEX  perpetual_positions_subaccountId_perpetualId_openEventId_index
ON perpetual_positions (subaccountId, perpetualId, openEventId DESC);
```

5. Asset Positions 테이블

테이블 정의

```sql
CREATE TABLE asset_positions (
id UUID PRIMARY KEY,
assetId VARCHAR NOT NULL REFERENCES assets(id),
subaccountId UUID NOT NULL REFERENCES subaccounts(id),
size DECIMAL NOT NULL,
isLong BOOLEAN NOT NULL
);

-- 인덱스
CREATE INDEX ON asset_positions (subaccountId);
CREATE INDEX ON asset_positions (subaccountId, assetId);
```

6. Candles 테이블 (시장 데이터)

테이블 정의

```sql
CREATE TABLE candles (
id UUID PRIMARY KEY,
startedAt TIMESTAMP NOT NULL,
ticker VARCHAR NOT NULL,
resolution ENUM('1MIN', '5MINS', '15MINS', '30MINS',
'1HOUR', '4HOURS', '1DAY') NOT NULL,
```

```
low DECIMAL NOT NULL,
high DECIMAL NOT NULL,
open DECIMAL NOT NULL,
close DECIMAL NOT NULL,
baseTokenVolume DECIMAL NOT NULL,
usdVolume DECIMAL NOT NULL,
trades INTEGER NOT NULL,
startingOpenInterest DECIMAL NOT NULL
);

-- 인덱스
CREATE INDEX ON candles (startedAt);
CREATE INDEX ON candles (ticker, resolution);
```

데이터 관계 및 무결성

1. 외래 키 관계

2. 참조 무결성 제약

```
-- 외래 키 제약 조건
ALTER TABLE orders
```

```sql
ADD CONSTRAINT fk_orders_subaccount
FOREIGN KEY (subaccountId) REFERENCES subaccounts(id);


ALTER TABLE fills
ADD CONSTRAINT fk_fills_subaccount
FOREIGN KEY (subaccountId) REFERENCES subaccounts(id);


ALTER TABLE fills
ADD CONSTRAINT fk_fills_order
FOREIGN KEY (orderId) REFERENCES orders(id);


-- 체크 제약 조건
ALTER TABLE orders
ADD CONSTRAINT check_good_til_exclusive
CHECK (
(goodTilBlock IS NOT NULL AND goodTilBlockTime IS NULL) OR
(goodTilBlock IS NULL AND goodTilBlockTime IS NOT NULL)
);
```

인덱싱 전략 및 성능 최적화

1. 쿼리 패턴 기반 인덱싱

서브계정별 조회 최적화

```sql
-- 서브계정의 모든 주문 조회
CREATE INDEX orders_subaccountId_status_index
ON orders (subaccountId, status);

-- 서브계정의 체결 내역 조회
CREATE INDEX fills_subaccountId_createdAt_index
ON fills (subaccountId, createdAt DESC);

-- 서브계정의 포지션 조회
CREATE INDEX perpetual_positions_subaccountId_status_index
ON perpetual_positions (subaccountId, status);
```

시장별 조회 최적화

```sql
-- 거래 쌍별 주문서 조회
CREATE INDEX orders_clobPairId_side_price_index
ON orders (clobPairId, side, price);

-- 거래 쌍별 최근 체결 조회
CREATE INDEX fills_clobPairId_createdAt_index
ON fills (clobPairId, createdAt DESC);
```

2. 부분 인덱스 활용

조건부 인덱싱

```sql
-- 활성 주문만 인덱싱
CREATE INDEX orders_active_clobPairId_price_index
ON orders (clobPairId, price)
WHERE status = 'OPEN';

-- TAKER 체결만 인덱싱 (거래량 분석용)
CREATE INDEX fills_taker_clobPairId_createdAt_index
ON fills (clobPairId, createdAt)
WHERE liquidity = 'TAKER';

-- 열린 포지션만 인덱싱
CREATE INDEX perpetual_positions_open_subaccountId_index
ON perpetual_positions (subaccountId, perpetualId)
WHERE status = 'OPEN';
```

3. 동시성 인덱스 생성

```sql
-- 서비스 중단 없이 인덱스 생성
CREATE INDEX CONCURRENTLY IF NOT EXISTS
orders_clobPairId_subaccountId_index
ON orders (clobPairId, subaccountId);
```

```
-- 기존 인덱스 안전 삭제
DROP INDEX CONCURRENTLY IF EXISTS old_index_name;
```

쿼리 최적화 방법

1. 복합 인덱스 활용

효율적인 범위 조회

```
-- 시간 범위 + 서브계정 조회
SELECT * FROM fills
WHERE subaccountId = $1
AND createdAt BETWEEN $2 AND $3
ORDER BY createdAt DESC;

-- 최적화된 인덱스
CREATE INDEX fills_subaccountId_createdAt_desc_index
ON fills (subaccountId, createdAt DESC);
```

정렬 최적화

```
-- 가격 순 정렬된 주문서 조회
SELECT * FROM orders
WHERE clobPairId = $1 AND side = 'BUY' AND status = 'OPEN'
```

```
ORDER BY price DESC, createdAt ASC;


-- 최적화된 인덱스
CREATE INDEX orders_clobPairId_side_status_price_createdAt_index
ON orders (clobPairId, side, status, price DESC, createdAt ASC);
```

2. 쿼리 실행 계획 최적화

EXPLAIN ANALYZE 활용

```
-- 쿼리 성능 분석
EXPLAIN (ANALYZE, BUFFERS)
SELECT o.*, f.size as fill_size
FROM orders o
LEFT JOIN fills f ON o.id = f.orderId
WHERE o.subaccountId = $1
AND o.createdAtHeight > $2;
```

통계 정보 업데이트

```
-- 테이블 통계 정보 갱신
ANALYZE orders;
ANALYZE fills;
ANALYZE perpetual_positions;
```

```
-- 자동 통계 수집 설정
ALTER TABLE orders SET (autovacuum_analyze_scale_factor = 0.05);
```

데이터 파티셔닝 및 아카이브 정책

1. 시간 기반 파티셔닝

Fills 테이블 파티셔닝

```
-- 월별 파티셔닝
CREATE TABLE fills_partitioned (
LIKE fills INCLUDING ALL
) PARTITION BY RANGE (createdAt);

-- 월별 파티션 생성
CREATE TABLE fills_2024_01 PARTITION OF fills_partitioned
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

CREATE TABLE fills_2024_02 PARTITION OF fills_partitioned
FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');
```

자동 파티션 관리

```
-- 파티션 자동 생성 함수
```

```
CREATE OR REPLACE FUNCTION create_monthly_partition(
table_name TEXT,
start_date DATE
) RETURNS VOID AS $$
DECLARE
partition_name TEXT;
end_date DATE;
BEGIN
partition_name := table_name || '_' || to_char(start_date, 'YYYY_MM');
end_date := start_date + INTERVAL '1 month';
EXECUTE format('CREATE TABLE %I PARTITION OF %I
FOR VALUES FROM (%L) TO (%L)',
partition_name, table_name, start_date, end_date);
END;
$$ LANGUAGE plpgsql;
```

2. 데이터 아카이브 전략

오래된 데이터 아카이브

```
-- 90일 이전 체결 데이터 아카이브
CREATE TABLE fills_archive (LIKE fills INCLUDING ALL);

-- 아카이브 프로시저
```

```
CREATE OR REPLACE FUNCTION archive_old_fills(
cutoff_date TIMESTAMP
) RETURNS INTEGER AS $$
DECLARE
archived_count INTEGER;
BEGIN
-- 아카이브 테이블로 이동
WITH moved_rows AS (
DELETE FROM fills
WHERE createdAt < cutoff_date
RETURNING *
)
INSERT INTO fills_archive
SELECT * FROM moved_rows;
GET DIAGNOSTICS archived_count = ROW_COUNT;
RETURN archived_count;
END;
$$ LANGUAGE plpgsql;
```

압축 및 저장 최적화

```
-- 아카이브 테이블 압축
ALTER TABLE fills_archive SET (
fillfactor = 100,
```

```
autovacuum_enabled = false
);

-- 컬럼 압축 설정
ALTER  TABLE fills_archive
ALTER COLUMN eventId SET  COMPRESSION lz4;
```

성능 모니터링 및 튜닝

1. 성능 메트릭 수집

쿼리 성능 모니터링

```
-- 느린 쿼리 식별
SELECT query, mean_time, calls, total_time
FROM pg_stat_statements
WHERE mean_time > 100
ORDER BY mean_time DESC;

-- 인덱스 사용률 확인
SELECT schemaname, tablename, indexname,
idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
```

```
WHERE idx_scan < 100;
```

테이블 크기 모니터링

```
-- 테이블 크기 조회
SELECT
schemaname,
tablename,
pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) as  size,
pg_size_pretty(pg_relation_size(schemaname||'.'||tablename)) as table_size,
pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename) -
pg_relation_size(schemaname||'.'||tablename)) as index_size
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;
```

2. 자동 튜닝 설정

연결 풀 최적화

```
// Knex.js 연결 풀 설정
const  knexConfig = {
client:  'postgresql',
connection: {
host:  process.env.DB_HOST,
```

```
port: process.env.DB_PORT,
database: process.env.DB_NAME,
user: process.env.DB_USER,
password: process.env.DB_PASSWORD,
},
pool: {
min: 10, // 최소 연결 수
max: 100, // 최대 연결 수
acquireTimeoutMillis: 30000, // 연결 획득 타임아웃
idleTimeoutMillis: 600000, // 유휴 연결 타임아웃
reapIntervalMillis: 1000, // 정리 주기
createRetryIntervalMillis: 200, // 재시도 간격
},
acquireConnectionTimeout: 30000,
asyncStackTraces: true,
};
```

자동 VACUUM 설정

```
-- 테이블별 VACUUM 설정 최적화
ALTER TABLE fills SET (
autovacuum_vacuum_scale_factor = 0.1,
autovacuum_analyze_scale_factor = 0.05,
```

```
autovacuum_vacuum_cost_delay = 10

);



ALTER  TABLE orders SET (

autovacuum_vacuum_scale_factor = 0.2,

autovacuum_analyze_scale_factor = 0.1

);
```

이러한 데이터베이스 스키마 설계와 최적화 전략을 통해 dYdX v4 Indexer는 높은 처리량과 낮은 지연시간을 제공하면서도 데이터 무결성과 일관성을 보장합니다.